

alongside the implementation. This is especially useful when enterprises use AI to modernise legacy platforms, upgrade libraries, migrate cloud services, or refactor monolithic systems.

Developing skills for SDD

Spec-driven development requires a different skill profile from traditional coding or casual AI prompting. Developers need engineering depth, but they also need the ability to express intent precisely. Product owners must learn to write acceptance criteria that are testable rather than aspirational. Architects must translate quality attributes into constraints that agents can respect. Testers must think in terms of executable evidence. Security specialists must define misuse cases, policy checks, and restricted actions early enough to influence implementation rather than waiting for end-stage review. Here is an example skill code snippet to understand this better:

```
---
name: spec-driven-development
description: >
  Use this skill whenever a user asks to
  build, add, or change a feature
  in a codebase. Enforces a spec-first
  workflow: write and confirm a
  structured spec before generating any
  code, then validate generated
  code against that spec. Trigger on
  requests like "build a feature",
  "implement X", "add support for Y", or
  "refactor Z" in a code project.
---

# Spec-Driven Development

## Workflow

1. **Specify** – Before writing code,
  draft a spec at
  `specs/<feature-slug>.md` with these
  sections:
  - `## Intent` – one paragraph, the
  "why"
  - `## Requirements` – numbered,
```

```
testable statements
  - `## Non-goals` – explicitly out of
  scope
  - `## Acceptance Criteria` –
  checklist, each mapped to a test

2. **Confirm** – Show the spec to the
  user. Do not generate code until
  they approve it or explicitly say to
  proceed.

3. **Plan** – Break the spec into an
  ordered task list. Each task must
  cite the requirement number(s) it
  satisfies.

4. **Generate** – Write code task-by-
  task. Reference the spec file
  in comments where non-obvious (`#
  satisfies REQ-3`).

5. **Validate** – After generating code,
  re-read the spec and check
  each Acceptance Criterion explicitly.
  Report PASS/FAIL per item –
  do not just say "looks good."

6. **Refine** – If a criterion fails or
  the user requests a change,
  update the spec FIRST, then
  regenerate the affected code. Never
  let code and spec drift silently.

## Rules
- Never skip straight to code generation
  for non-trivial features.
- Never mark an acceptance criterion as
  passed without checking it.
- Keep the spec file updated – it should
  always describe current
  intended behavior, not just the
  original request.
```

A useful SDD skill path begins with requirement articulation. Practitioners should learn to separate business goals, user journeys, functional behaviour, constraints, and assumptions. They should also become comfortable writing non-goals, because stating what the system will not do prevents agents from adding unnecessary features. The

next skill is decomposition. A good specification must be broken into small, bounded tasks that can be implemented, reviewed, and tested independently. This reduces the risk of large AI-generated changes that are difficult to inspect.

Another important skill is evaluation design. Teams must define how they will know that the implementation is correct. This includes unit tests, integration tests, contract tests, performance checks, accessibility checks, security scans, policy validations, and manual review criteria. In agentic development, evaluation is not a final stage; it is the steering mechanism that keeps the agent aligned. Finally, practitioners need AI collaboration literacy: how to provide context, how to bound tasks, how to inspect generated diffs, how to challenge assumptions, how to preserve institutional knowledge, and how to decide when human engineering judgment must override AI output.

Governance model in spec-driven development

Governance in SDD should not be designed as a bureaucratic checkpoint that slows every team. It should function as a lightweight control system that makes safe delivery easier. A practical governance model begins with specification ownership. Every specification should have named business, architecture, engineering, quality, and security owners depending on the risk level of the work. Their role is not to write every detail, but to ensure that the specification is complete enough for reliable implementation and review.

The second governance layer is risk classification. Not every feature needs the same level of formality. A temporary prototype may need a short intent brief. A customer-facing financial transaction, healthcare workflow, identity service, or production infrastructure change requires a stronger specification with security, privacy, audit, rollback, and approval